

Lezione 8: Esercitazione Pratica

Collezioni, Interfacce Funzionali, Generics, Stream e Design Pattern

Michele Gentile

28 Maggio 2026

1 Logistica di una Flotta Stellare

1. Si definisca una classe astratta **Astronave** dotata di un **codiceIdentificativo** (String), un livello di **scudoEnergia** (int) e una **massa** (double).
2. Si creino due sottoclassi specifiche: **CacciaIntercettore** (dotata di un numero di siluri laser) e **IncrociatoreCargo** (dotata di una capacità massima di carico in tonnellate).
3. Si implementi una classe generica **Hangar** che gestisce un insieme di astronavi dello stesso tipo utilizzando internamente un **HashSet**. Fornire i metodi per inserire un'astronave e rimuoverla tramite codice identificativo.
4. Implementare correttamente **equals** e **hashCode** nella classe *Astronave* per garantire che non vi siano navi duplicate con lo stesso codice identificativo all'interno dell'hangar.
5. Si doti la classe *Hangar* di un metodo **ottieniNaviPronte()** che, tramite l'uso delle **API Stream**, restituisca una lista di tutte le astronavi con uno scudo energetico strettamente superiore a 75, ordinate in modo decrescente per massa.

🔗 Obiettivo: Generics vincolati, Stream API e Meccanismi di Unicità

L'obiettivo è padroneggiare la programmazione generica con vincoli di tipo (*bounded generics*) e saper manipolare collezioni complesse attraverso pipeline di Stream per operazioni di filtraggio e ordinamento personalizzato.

💡 Tip: Ereditarietà nei Generics

Ricordate che un **Hangar** non può ospitare una flotta mista se si vuole specializzare l'hangar per soli caccia. Usando **Hangar** si vincola il tipo staticamente, ma i metodi interni di stream ereditano tutte le proprietà della classe astratta *Astronave*.

2 Bacheca delle Missioni della Gilda

1. Si modelli una classe **Missione** caratterizzata da un **titolo**, una **difficoltà** (livello da 1 a 10) e una **ricompensa** in monete d'oro.

2. Si applichi il **Design Pattern Observer**: si definisca un'interfaccia **Avventuriero** dotata del metodo `void riceviNotifica(Missione m)` e una classe **BachecaGilda** che mantiene un registro di avventurieri iscritti.
3. Ogni volta che una nuova missione viene pubblicata sulla *BachecaGilda*, tutti gli avventurieri iscritti devono essere notificati.
4. Si doti la classe *BachecaGilda* di un metodo avanzato di sottoscrizione: `void iscrivi(Avventuriero a, Predicate filtro)`. La bacheca deve notificare l'avventuriero *solo ed esclusivamente* se la missione soddisfa i criteri definiti dall'interfaccia funzionale **Predicate**.
5. Si crei una classe di test in cui un avventuriero "Guerriero" accetta solo missioni con difficoltà superiore a 7, mentre un avventuriero "Ladro" accetta solo missioni con ricompensa superiore a 500 monete, usando le **espressioni lambda**.

🔗 Obiettivo: Design Pattern Observer e Interfacce Funzionali

L'obiettivo è disaccoppiare la logica di notifica di un sistema ad eventi introducendo filtri dinamici iniettati a runtime tramite costrutti di programmazione funzionale.

⚠️ Attenzione: Modifiche concorrenti durante la notifica

Prestate attenzione a non rimuovere o inserire osservatori dalla lista degli iscritti mentre state iterando su di essa per inviare le notifiche, al fine di evitare una `ConcurrentModificationException`.

3 Infiltrazione nella Rete Cyberpunk

1. Si realizzi una classe **ServerRete** che rappresenta un terminale informatico. Ogni server ha un **indirizzoIP** (`String`) e un flag booleano **contieneFirewall**.
2. Ciascun *ServerRete* fornisce un collegamento ad altri nodi della rete, modellato tramite una collezione `List connessioni`.
3. Si realizzi una classe **Netrunner** che possiede un metodo booleano `propagaMalware(ServerRete nodoIniziale, ServerRete nodoTarget)`. Il metodo deve cercare un percorso dal nodo iniziale al nodo target per infettarlo.
4. Il malware può propagarsi da un server all'altro solo se il server corrente non contiene un firewall attivo. La ricerca deve essere implementata tramite una strategia **ricorsiva con backtracking**.
5. **Avanzato**: Per simulare le tracce lasciate nella rete (analogo al "filo di Arianna"), il metodo deve memorizzare e stampare l'esatta sequenza di indirizzi IP visitati per raggiungere l'obiettivo, garantendo inoltre di evitare cicli infiniti nella rete utilizzando un opportuno **Set** di nodi già visitati.

🔗 Obiettivo: Strutture Dati a Grafo, Ricorsione e Backtracking

L'obiettivo è applicare concetti algoritmici di esplorazione di strutture dati collegate (grafi) combinandoli con l'uso efficiente delle collezioni per il tracciamento degli stati ed evitare loop di computazione.

💡 Tip: Gestione dei Cicli

In una rete di server non gerarchica, due nodi possono essere connessi vicendevolmente. Prima di effettuare la chiamata ricorsiva sui nodi adiacenti, inserite il nodo corrente in un `HashSet` di nodi visitati. Se il nodo è già presente, interrompete quel ramo di computazione!

4 Simulatore di Arena Fantasy

1. Si crei una classe **Combattente** caratterizzata da un **nome**, **puntiVita** (int) e una strategia di attacco corrente.
2. Si implementi il **Design Pattern Strategy**: si definisca l'interfaccia **StrategiaAttacco** dotata del metodo `int eseguiColpo(Combattente attaccante, Combattente difensore)`.
3. Si implementino tre strategie distinte:
 - **AttaccoFurtivo**: infligge un danno fisso ma raddoppia se i punti vita del difensore sono al massimo.
 - **MagiaArcana**: infligge un danno proporzionale al numero di lettere che compongono il nome dell'attaccante.
 - **FuriaGuerriera**: infligge più danni quanti meno sono i punti vita residui dell'attaccante stesso.
4. Si modelli una classe **Arena** che contiene un insieme di combattenti in una **PriorityQueue**. La coda deve mantenere i combattenti ordinati in base ai punti vita crescenti (chi ha meno vita ha la priorità di mossa).
5. Si scriva un metodo `avviaTurno()` in cui il primo combattente della coda attacca un avversario estratto casualmente dalla coda stessa, mutando dinamicamente la propria strategia di attacco se i suoi punti vita scendono sotto il 30

🔗 Obiettivo: Design Pattern Strategy e PriorityQueue

L'obiettivo è comprendere come isolare e rendere intercambiabili gli algoritmi di comportamento di un oggetto a runtime, gestendo la coordinazione degli attori mediante code di priorità basate su comparatori personalizzati.